DSS-MS Unmasked: Public Cross-Join Sanitizer-Key Recovery and Fresh-Message Forgery

Martin Feussner
Selmer Center, University of Bergen
martin.feussner@uib.no

18 October 2026

This audit used the OpenAI model Daybreak Blue at the ultra reasoning level. OpenAI Codex generated and selected the paper title and deliberately chose "Unmasked" as wordplay on the DSS-MS public sanitizer-key mask revealing the sanitizer key. OpenAI Codex performed AI-assisted paper and source reconstruction, cryptanalysis, implementation, experimentation, review coordination, and technical writing. A separately tasked AI reviewer checked the sealed primary evidence for internal consistency and returned `PASS`. A fresh-context AI agent independently reconstructed the target, executed eight prospectively sealed full-profile trials, and returned `PASS_INDEPENDENT_REPRODUCTION`. A separately tasked hostile falsification review returned `PASS_FINAL_FALSIFICATION_REVIEW`, and the final exact-target history recheck returned `PASS_FINAL_EXACT_TARGET_HISTORY_GATE`. An independent AI Publication Reviewer reviewed the corrected package and returned `PASS_PUBLICATION_REVIEW`. Independent human verification was not performed; it is optional and is not a publication requirement.

Abstract

DSS-MS, proposed in IACR ePrint 2026/041, publishes a Chinese-remainder sanitizer-key mask $S_x = A\,sk_s$ as an unreduced integer. The dynamic Join procedure recomputes the CRT coefficient after adding sanitizers and publishes $S'_x = A'\,sk_s$ under the same global sanitizer public key $PK_s = sk_s P$. A passive observer therefore obtains

$$\gcd(S_x, S'_x) = sk_s \gcd(A, A').$$

On the first and only precommitted exact primary instance, containing 3,000 initial sanitizers and one ordinary addition of 20, the residual GCD was one. One public integer GCD recovered the 251-bit global sanitizer scalar, and the public equation $sk_s P = PK_s$ validated the candidate.

Given one valid signature (R, z, s, T) on full message (m_0, m), the recovered scalar permits the observer to preserve (R, z), retarget the chameleon-hash component, and obtain a signature accepted on the distinct full message (m_0, m'). Only the sanitizable component changes. The sealed primary standard-library implementation used actual BLS12-381 G1 arithmetic, matched the pinned source's SHA-256 transcript conversion, and called a source-equivalent reconstruction of the ordinary verifier. Its exact run took 34.807209 seconds and 20,480 KiB peak resident memory with one worker. Three attack-specific negative controls rejected.

An independent implementation ran eight prospectively sealed exact profiles. Seven recovered exactly after one Join and forged successfully. The retained eighth trial had residual factor 2 after one Join and at 3,040 sanitizers; its cumulative public-mask GCD became exact at 3,060 sanitizers, where the forgery and controls passed. The full eight-trial run took 372.77 seconds and 27,136 KiB peak resident memory with one worker. Thus one-Join recovery is practical but not guaranteed. The attack recovers the global sanitizer scalar rather than the original signer scalar, changes only the sanitizable component, and excludes Revoke, the separate range-condition correctness defect, and any compiled-author-binary claim. Repeated-common-factor GCD recovery is prior art in the PA-CRT lineage; we claim no novelty for the technique. Final falsification, exact-target history review, and independent Publication Review passed.

1 Introduction and exact claim

Zeng, Zhang, Cui, Zhong, and Wang propose a dynamic sanitizable signature with multiple sanitizers (DSS-MS) for privacy-preserving shared logistics [1]. A signer authenticates a message $M = (m_0, m)$, where m_0 is fixed and m is designated sanitizable. Every authorized sanitizer receives its own secret sk_i, while the system uses one global temporary sanitizer scalar sk_s. A public CRT mask lets each sanitizer derive sk_s by reduction modulo its own secret. The scheme adds and revokes sanitizers dynamically.

The addition lifecycle republishes the CRT mask while keeping the same global scalar. This produces two public integers with a common secret factor. The exact claim is:

> For the paper-claimed BLS12-381/SHA-256 profile with 3,000 existing sanitizers, a passive observer retains the public S_x values immediately before and after one ordinary Join(20). On the precommitted primary instance, their GCD is exactly sk_s. The observer validates this value against PK_s, then uses one valid signature to produce an accepted signature on a fresh full message whose sanitizable component is distinct.

The result is design-level: both the paper and pinned source use unreduced integer masks and reuse the scalar through Join. It needs no parser behavior, malformed point, weak curve, secret channel, attacker-selected enrollment, rollback, side channel, or implementation memory defect. The recovered value is not the original signer's Schnorr secret. It is the global sanitizer scalar, an equivalent signing capability for the scheme's designated message transformation.

The primary experiment, evidence consistency review, fresh-context independent reproduction, final falsification review, final exact-target history recheck, and independent Publication Review passed. The result is classified as a verified practical design attack under the project charter.

Contributions. We identify the common-factor exposure across ordinary public-parameter epochs; prove the exact key-recovery identity and its public validation rule; give the fresh-message signing bridge through the scheme's own sanitization equations; report a precommitted full-profile execution and eight-trial independent reproduction with controls; state the source and verifier boundary; distinguish a separate correctness defect; credit the PA-CRT GCD lineage; and give repair directions.

2 Target, version, source, and profile

The target is the current landing-visible body of *Towards Privacy-Preserving Unmanned Aerial Vehicles Shared Logistics via Dynamic Sanitizable Signature with Multiple Sanitizers*, IACR ePrint 2026/041 [1]. The retained 15-page PDF has SHA-256 digest

`d35dcd98dc2442b6e263ef58276086832287a3f9d2f9e0131e4b0ad6c7324a20`.

The official landing page records receipt and approval on 11 January 2026. A version-history endpoint was unavailable during the audit, so this report does not assign an unsupported revision ordinal.

The paper links the authors' DSS-MS repository [2]. The selected paper-time source commit and tree are

`b994a3f46cbe5501f47f53aa2e6942cff714b53a`,
`5d1b072040170d6df8bca709403d1e01d609536e`.

At the retrieved head `b48e6359a0c3dce11cc6bf8445577770418d6d27`, the relevant `DSS/src/DSS.cpp` remained byte-identical to its paper-time form. The source fixes 3,000 sanitizers and 256-bit sanitizer values. The paper evaluates batch additions including 20, and both paper and source accept the addition count as an input to Join. We therefore freeze profile

`DSSMS-BLS12381-SHA256-N3000-ADD20-CRT-INMEM-20260111`.

The source uses BLS12-381 and SHA-256. It provides no fixed portable known-answer vector or normative wire format. Our endpoint is therefore a source-equivalent reconstruction of the ordinary equations and transcript conversion, not execution of a compiled author binary.

3 DSS-MS equations

3.1 Public CRT mask

Let the sanitizer secrets $sk_1, \ldots, sk_n$ be pairwise coprime positive integers. Define

$$N = \prod_{i=1}^{n} sk_i, \qquad N_i = N/sk_i, \tag{1}$$

$$y_i N_i \equiv 1 \pmod{sk_i}, \qquad A = \sum_{i=1}^{n} y_i N_i. \tag{2}$$

The construction samples a global sanitizer scalar $sk_s \in \mathbb{Z}_q^*$, publishes

$$S_x = A\, sk_s \quad \text{over the integers,} \qquad PK_s = sk_s P, \tag{3}$$

and gives sanitizer i the intended recovery rule $sk_s = S_x \bmod sk_i$. Because $A \equiv 1 \pmod{sk_i}$, that rule returns $sk_s \bmod sk_i$. Literal recovery therefore additionally requires $sk_s < sk_i$; Section 7 separates this omitted correctness condition from our attack.

The public nature of S_x is explicit. KGen returns it, the paper's unforgeability experiment gives it to the adversary, the deployment text says the logistics company publishes it, and the public parameters contain it. The source stores it as a GMP integer and serializes its complete hexadecimal representation.

3.2 Join lifecycle

Suppose an ordinary Join adds r sanitizers. It forms the enlarged modulus set, recomputes Equation (2) as A', and publishes

$$S'_x = A'\, sk_s. \tag{4}$$

The named scalar remains sk_s, and PK_s is not rotated. The pinned source corroborates the paper: KGen assigns `pp.u_s = keyPair.sk * u`, while Join receives the same sanitizer scalar and assigns `pp.u_s = sk_san * crt.u`. Neither operation applies a modulus to this multiplication.

3.3 Signing and verification

Let P generate a prime-order group of order q. A signature is $\sigma = (R, z, s, T)$. On $M = (m_0, m)$, the ordinary verifier computes

$$e = H_0(m, T), \tag{5}$$

$$CH = T + sP + ePK_s, \tag{6}$$

$$c = H_1(m_0, R, CH), \tag{7}$$

and accepts exactly when

$$zP = R + cPK, \tag{8}$$

where PK is the original signer's public key. An authorized sanitizer changes m by preserving CH and hence the signer's challenge.

4 Public cross-Join key recovery

Theorem 1 (Common-factor recovery)**.** *For public masks $S_x = A\, sk_s$ and $S'_x = A'\, sk_s$ computed over the integers with the same positive scalar sk_s,*

$$\gcd(S_x, S'_x) = sk_s \gcd(A, A').$$

If $\gcd(A, A') = 1$, the GCD is exactly sk_s.

Proof. For positive integers, $\gcd(au, av) = a \gcd(u, v)$. Substitute $a = sk_s$, $u = A$, and $v = A'$. The coprime case follows immediately. □

The observer need not know any sanitizer secret or either coefficient. It computes $g = \gcd(S_x, S'_x)$ directly. The public point supplies an exact candidate test:

$$gP \stackrel{?}{=} PK_s. \tag{9}$$

If a residual factor $d = \gcd(A, A') > 1$ remains, the one-Join candidate is $sk_s d$ and normally fails Equation (9). Further public epochs can reduce the common residual by taking a GCD across all retained masks. The primary experiment used exactly one Join and had unit residual. The independent experiment below measured both outcomes: seven of eight sealed trials had unit residual after one Join, while one retained residual required two more public epochs. One-Join recovery is therefore not asserted to be universal.

5 From the sanitizer scalar to a fresh-message forgery

Theorem 2 (Retargeting)**.** *Suppose (R, z, s, T) is accepted on (m_0, m) under (PK, PK_s), and an attacker knows sk_s such that $PK_s = sk_s P$. For any distinct sanitizable component m' and any $k \in \mathbb{Z}_q$, define*

$$T' = CH - kP, \tag{10}$$
$$e' = H_0(m', T'), \tag{11}$$
$$s' = k - e' sk_s \pmod q. \tag{12}$$

Then (R, z, s', T') is accepted by the ordinary verifier on the fresh full message (m_0, m').

Proof. Using $PK_s = sk_s P$ and Equations (10)–(12), the verifier reconstructs

$$\begin{aligned} T' + s'P + e'PK_s &= CH - kP + (k - e'sk_s)P + e'sk_s P \\ &= CH. \end{aligned}$$

It therefore obtains the same value in Equation (7). Since (R, z) is unchanged, the already valid Equation (8) remains valid. Because $m' \neq m$, the full message is distinct. □

This is the scheme's ordinary sanitization transformation performed by a party that received no sanitizer secret. One valid signature is required. No sanitizing-oracle query is used. The fresh full message changes only the field that DSS-MS declares sanitizable; changing m_0 is outside this claim.

6 Primary full-profile reproduction

The primary experiment was sealed before execution with decimal seed `2026041`. The command fixed $n = 3000$, an addition of 20, 256-bit sanitizer draws, and 24 explicitly non-full-profile proxy trials. No seed was discarded or selected after observing the residual GCD.

The dependency-free Python implementation generates source-shaped pairwise coprime sanitizer values, computes the exact unreduced CRT sums, and performs the public attack. Its signature layer implements affine arithmetic on actual BLS12-381 G1 and checks that the generator has prime order. Transcript hashing matches the pinned source's relevant path: minimal big-endian GMP message bytes, MIRACL's 49-byte compressed G1 encoding, and a 32-byte SHA-256 digest loaded into the high end of a zeroed 48-byte integer before reduction modulo q.

The attack function receives the two public epochs, the public keys, the public base message and signature, the selected fresh message, and attacker scalar k. Its interface contains no sk_i or hidden sk_s argument. All tested points lie on the curve and in the prime-order subgroup. The three rejecting controls distinguish the result from unconditional verifier acceptance, an irrelevant message mutation, and key-independent retargeting.

The checked-in public package offers two modes. A quick 96-sanitizer mode exercises the full algebraic, curve, verifier, and control path. The exact mode replays the deterministic 3,000-sanitizer, Join(20) primary instance. Neither is represented as the fresh-context independent gate.

Table 1: Sealed primary exact-profile result.

Field	Result
Initial / added / final sanitizers	3,000 / 20 / 3,020
Old / joined product size	763,995 / 769,080 bits
Recovered public GCD size	251 bits
Residual $\gcd(A, A')$	1
Recovered scalar satisfies $gP = PK_s$	yes
Honest signature accepted before and after Join	yes
Fresh full message distinct	yes
Forgery preserves (R, z) and reconstructs the same CH	yes
Forged fresh message accepted	yes
Mutation without retargeting accepted	no
Wrong-scalar retargeting accepted	no
Forgery under wrong PK_s accepted	no
Wall time / peak resident memory / workers	34.807209 s / 20,480 KiB / 1

6.1 Fresh-context independent reproduction

A fresh-context agent derived a clean-room standard-library implementation from the frozen paper and source before inspecting the primary reproducer. Its plan, code, commands, and eight exact 32-byte seeds were prospectively sealed. Every fixed seed was run once and retained. Each trial began with 3,000 sanitizers and an ordinary $\mathsf{Join}(20)$. A nonunit residual was fixed in advance to remain a one-Join failure, with at most three further ordinary public epochs permitted as a separately classified cumulative-GCD test.

Table 2: Fresh-context independent full-profile results.

Trial	One-Join residual	One-Join result	Runtime (s)	Terminal result
1	1	PASS	42.548	PASS one Join
2	1	PASS	42.080	PASS one Join
3	1	PASS	41.115	PASS one Join
4	1	PASS	41.226	PASS one Join
5	1	PASS	41.609	PASS one Join
6	2	retained failure	70.129	PASS at 3,060
7	1	PASS	41.249	PASS one Join
8	1	PASS	41.929	PASS one Join

All seven one-Join exact recoveries passed Equation (9), fresh full-message verification, message-mutation rejection, wrong-key rejection, honest before/after-Join checks, and curve/subgroup checks. Trial 6 retained residual 2 at 3,020 and 3,040 sanitizers, so no forgery was attempted with its invalid candidate. The cumulative residual became one at 3,060; public-key validation, the fresh-message forgery, and all controls then passed. It remains a one-Join failure and a separately classified multi-epoch success.

The observed one-Join exact-recovery rate is 7/8, an empirical result for fixed seeds rather than an exact probability. The complete eight-trial run, including the two extra epochs for trial 6, took 6 minutes 12.77 seconds and 27,136 KiB peak resident memory with one worker. The independent manifest has SHA-256

`e94ba0b60d291b6ce3136e995f1459dc3fb28147de33908f63cb1c61cb4235e2`.

The independent source, result stream, report, and verdict have respective SHA-256 digests

`443b0444f58bf2daf4a0a5951087cf77258a844a49c2b84e2aa115cfacb69aaf`,
`1a3cd768bcd3106db0f82edc338064faf6118ae574c2b29a6fe8c82a2e087448`,
`f442e1f684aaf5707bf651e45d5ba2467cd8fecbf3d1b369bff8df6fa960c671`,
`83d498952d5c88d96202ada5eb52c442ed3a68575323b732dfde76b3267477ca`.

The sealed verdict is `PASS_INDEPENDENT_REPRODUCTION`.

7 Scope and limitations

The following qualifications are part of the result.

- The primary one-Join attack passively retains two public S_x epochs around one ordinary Join(20). It does not choose enrolled keys or access a secret delivery channel. The independent cumulative variant retained further ordinary public epochs only after recording a one-Join failure.
- The endpoint uses one valid signature. The forged full message changes only its sanitizable component, from (m_0, m) to (m_0, m').
- The recovered value is the global sanitizer scalar sk_s, not the original signer scalar. It nevertheless supplies the intended sanitization signing capability to an unauthorized observer.
- The quantitative evidence is one precommitted exact primary instance plus eight prospectively sealed independent exact trials. Seven of eight independent trials recovered after one Join; the sole residual trial recovered after two further public epochs. This does not prove an exact success probability or a universal epoch bound. The primary small proxy trials and random-integer heuristic are not full-profile success-rate evidence.
- Verification is a source-equivalent reconstruction of the ordinary paper and pinned-source equations. The audit did not compile and run the author binary and makes no normative codec claim.
- The paper and source omit $sk_s < \min_i sk_i$, so not every honest sanitizer necessarily recovers the literal scalar by one reduction. That separate correctness defect is excluded from the attack.
- The paper and source differ on Revoke. Revoke is excluded; the candidate uses only the paper- and source-consistent KGen-then-Join lifecycle.
- The attack-history result is a bounded search through 18 October 2026, not proof of absolute priority. The final exact-target recheck passed; private, removed, or unindexed disclosures still cannot be excluded.

8 Prior method and attack history

This report makes no novelty claim for repeated-common-factor recovery by an integer GCD. DSS-MS directly cites PA-CRT [3]. Tao et al.'s B-DSPA work describes public CRT-related products with a reused hidden factor [4]; Kabil, Aslan, and Azer record that GCD method, analyze its lifecycle assumptions, and discuss flaws in parts of the preceding PA-CRT cryptanalysis [5]. These works establish relevant technique ancestry. They do not name DSS-MS, analyze $S_x = A\,sk_s$, recover the DSS-MS sanitizer scalar, or reach its ordinary signature verifier.

The bounded exact-target search covered the title, acronym, ePrint identifier, authors, repository history and issue surfaces, corrections, errata, sanitizer-key recovery, Join, common factors, GCD, and forgery terms. It located no earlier public attack against DSS-MS as of 18 October 2026. The scheme is recent and bibliographic indexing can lag. We therefore claim only that no prior public exact-target attack was located in the recorded search. The final recheck returned `PASS_FINAL_EXACT_TARGET_HISTORY_GATE`. It confirmed that the official landing and attack-relevant source were unchanged and again found no exact-target disclosure. The public GCD work against PA-CRT and B-DSPA remains credited as technique ancestry, and no technique novelty is claimed.

9 Repair direction

The public update mechanism must not emit multiple ordinary integers sharing the same secret scalar as a factor. One design direction is to sample an independent sanitizer scalar and matching public key for each epoch, bind the epoch and key into signatures, and give verification explicit version semantics. Another is to replace the public CRT mask with a conventional recipient-specific key-establishment mechanism so that public lifecycle updates do not encode one capability as a reusable common factor.

A revision should also state and enforce the scalar/modulus range needed by the intended residue recovery, reconcile Join and Revoke, and define a normative transcript and parameter-transition policy. These are repair directions rather than a validated patch. Any concrete revision needs fresh correctness, compatibility, and security analysis.

10 Conclusion

DSS-MS republishes an unreduced CRT sanitizer-key mask after adding sanitizers, while retaining the same global sanitizer scalar and public key. The two public epochs therefore expose that scalar through a standard common-factor GCD whenever their CRT coefficients are coprime. The one precommitted exact primary instance had unit residual, recovered the 251-bit scalar in a 34.807209-second end-to-end run, and used one valid signature to produce an ordinary-verifier-accepted signature on a fresh full message. The evidence is limited to the stated Join lifecycle, message transformation, and source-equivalent verifier boundary. Fresh-context independent reproduction passed on all eight fixed trials, with seven one-Join successes and one retained residual that recovered through cumulative public epochs. Final falsification, exact-target history review, and independent Publication Review also passed.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

References

[1] M. Zeng, Q. Zhang, J. Cui, H. Zhong, and F. Wang, "Towards Privacy-Preserving Unmanned Aerial Vehicles Shared Logistics via Dynamic Sanitizable Signature with Multiple Sanitizers," IACR Cryptology ePrint Archive, Report 2026/041, 2026. https://eprint.iacr.org/2026/041

[2] M. Zeng et al., "DSS-MS author implementation," paper-time commit b994a3f46cbe5501f47f53aa2e6942cff714b53a. https://github.com/rookie-papers/DSS-MS

[3] J. Zhang, J. Cui, H. Zhong, Z. Chen, and L. Liu, "PA-CRT: Chinese Remainder Theorem Based Conditional Privacy-Preserving Authentication Scheme in Vehicular Ad-Hoc Networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 2, pp. 722–735, 2021. https://doi.org/10.1109/TDSC.2019.2904274

[4] Tao et al., "B-DSPA: A Blockchain-Based Dynamically Scalable Privacy-Preserving Authentication Scheme in Vehicular Ad-Hoc Networks," *IEEE Internet of Things Journal*, vol. 11, no. 1, p. 1385 ff., 2024. https://doi.org/10.1109/JIOT.2023.3289057

[5] A. M. Kabil, H. Aslan, and M. Azer, "Cryptanalysis of Two Conditional Privacy Preserving Authentication Schemes for Vehicular Ad Hoc Networks," *Cryptography*, vol. 8, no. 1, article 4, 2024. https://doi.org/10.3390/cryptography8010004